

Lecture 1

Introduction & Recursion

Dr. Yusuf H. Sahin
Istanbul Technical University

sahinyu@itu.edu.tr

Computers and Modelling

- **Program:** A description of how the information is manipulated.
- **Programming Language:** A formal, artificial language to define a program by instructions. It is used to write the *software* which controls the behavior of the *hardware*.

Example: Perceptron. A 5-ton computer which classifies punch cards marked on the left and punch cards marked on the right.



Prof. Rosenblatt's «perceptron» machine (1958)

$$v = \sum_{i=1}^m w_i x_i + b$$

```
import numpy as np
np.multiply(x,W) + b
```

Python implementation of the perceptron

A Selected Idiom: x^2

- We could investigate the differences between programming languages by checking the *idioms*.

Pascal

```
Function square(x: Integer): Integer;  
Begin  
    Result := x*x;  
End;
```

Python

```
def square(x):  
    return x*x
```

C++

```
int square(int x){  
    return x*x;  
}
```

<https://programming-idioms.org/all-idioms>

Lisp

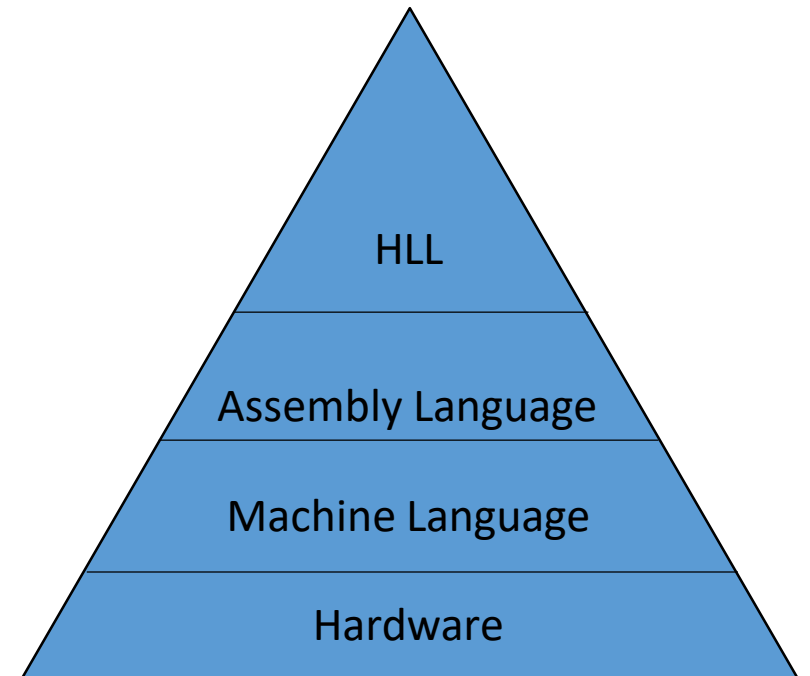
```
(defun square (x)  
  (* x x))
```

Haskell

```
square x = x * x
```

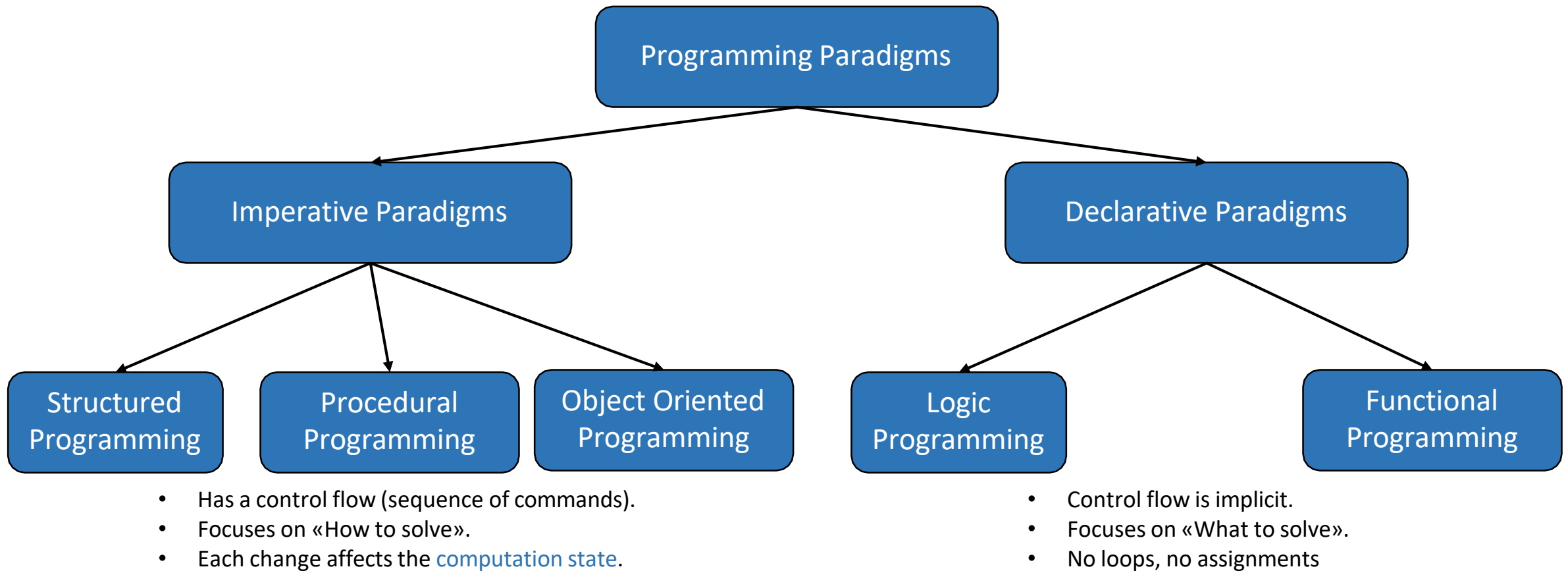
Why Programming Languages?

- Older programs were able to use the hardware directly.
- Modern languages focus on the confidence of the programmer: A **higher level**.
- A programming language runs on the computer thanks to an implementation, a program to run HLL programs.



Programming Paradigms

Paradigm: A style of programming



Imperative Paradigms

- Imperative paradigm uses assignments and loops like the [Turing Machine](#).

Structured Programming

- [Böhm–Jacopini theorem](#): Any computable function can be implemented using three «structures»: Sequence, selection, iteration
- The term is dedicated to Edsger Wybe Dijkstra because of his work «Notes on Structured Programming»

Procedural Programming

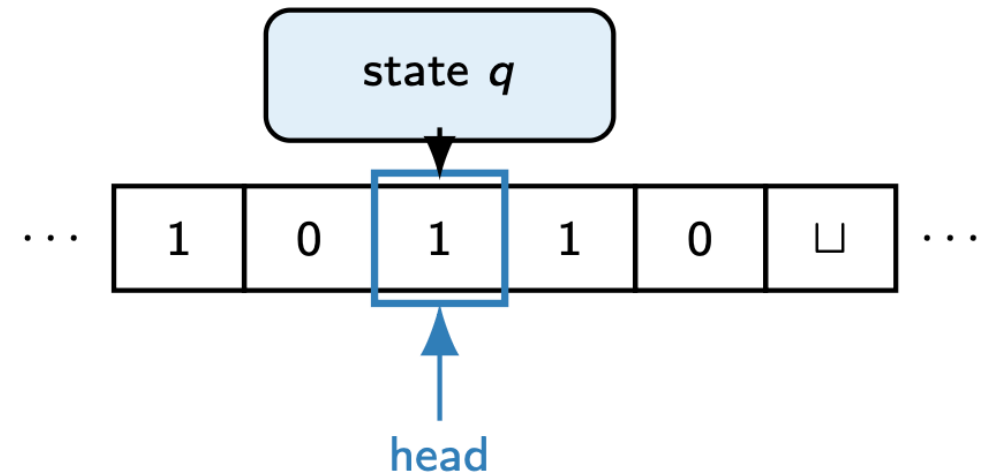
- The program is divided into procedures (functions).
- C is a pure procedural programming language.

Object Oriented Programming

- Objects which have encapsulated states communicates by messaging each other.
- C# is a pure OOP language.

Remember: Turing Machine

- A Turing Machine is an abstract model of computation. It consists of:
 - an infinite tape,
 - a read/write head,
 - a finite set of states,
 - transition rules.
- At each step, the machine:
 - reads the current symbol,
 - writes a new symbol,
 - moves left or right,
 - changes its state.
- Computation proceeds one small step at a time.

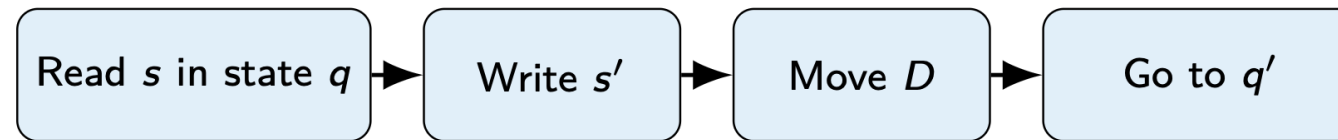


The machine reads, writes, moves, and changes state.

One Step of Computation

- A transition rule tells the machine what to do next.

$$\delta(q, s) = (q', s', D), \quad D \in \{L, R\}$$



- Imperative programs describe computation as a sequence of state changes.

tape $\approx$ memory
symbol update $\approx$ assignment
state $\approx$ control state
transition $\approx$ next command

Declarative Paradigms

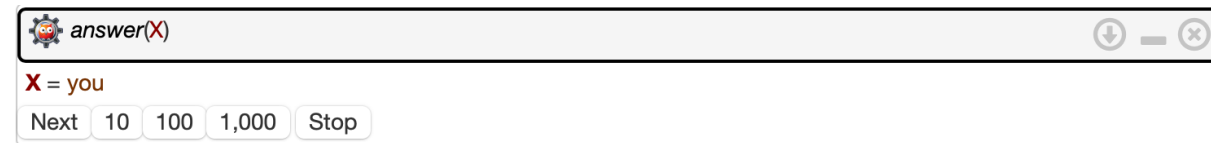
- The programmer directly specifies the desired result. The algorithm to obtain the result is implicitly takes part in the programming language.

<https://swish.swi-prolog.org>

Logic Programming

- The programmer states the facts and rules. Then, inputs a logically solvable question to the system.

```
1 % Facts
2 mother(mary, you).
3 mother(mary, alice).
4 mother(mary, bob).
5
6 % Rule: X and Y are siblings if they have the same mother
7 % and they are different people.
8 sibling(X, Y) :-
9     mother(M, X),
10    mother(M, Y),
11    dif(X, Y).
12
13 % Question:
14 % Who was born from your mother, but is not your sibling?
15 answer(X) :-
16     mother(M, you),
17     mother(M, X),
18     \+ sibling(X, you).
```



:- Rule
\+ Not provable

Logic Programming Example

% Direct prerequisite facts

```
requires(functional_programming, programming).  
requires(functional_programming, discrete_math).  
requires(advanced_functional_programming,  
functional_programming).  
requires(advanced_functional_programming, data_structures).  
requires(machine_learning, linear_algebra).  
requires(deep_learning, machine_learning).
```

% Student facts

```
passed(alice, programming).  
passed(alice, discrete_math).  
passed(alice, data_structures).  
passed(bob, programming).  
passed(bob, discrete_math).  
passed(charlie, linear_algebra).  
passed(charlie, machine_learning).
```

% Base case:

% P is a prerequisite of Course if Course directly requires P.

```
prerequisite(Course, P) :-  
    requires(Course, P).
```

```
prerequisite(Course, P) :-  
    requires(Course, Mid),  
    prerequisite(Mid, P).
```

```
can_take(Student, Course) :-  
    \+ (  
        prerequisite(Course, P),  
        \+ passed(Student, P)  
    ).
```

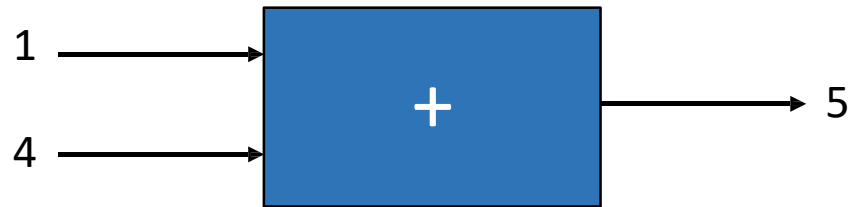
Functions

Functional Programming

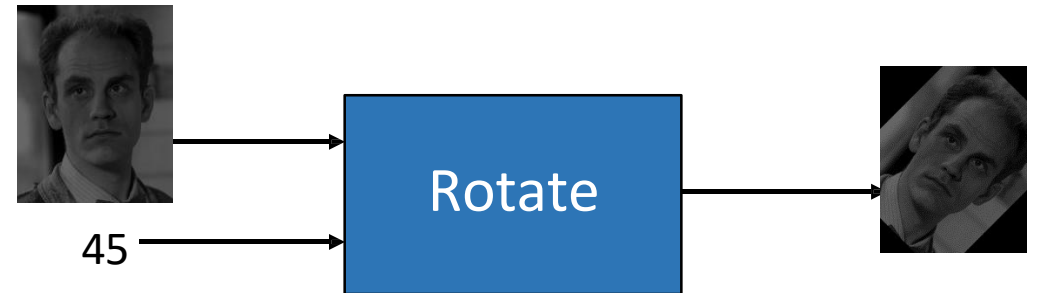
- The control flow is formed by combining function calls.
- Functional programming is based on Lambda calculus.

- A procedure which outputs a result according to the input(s).

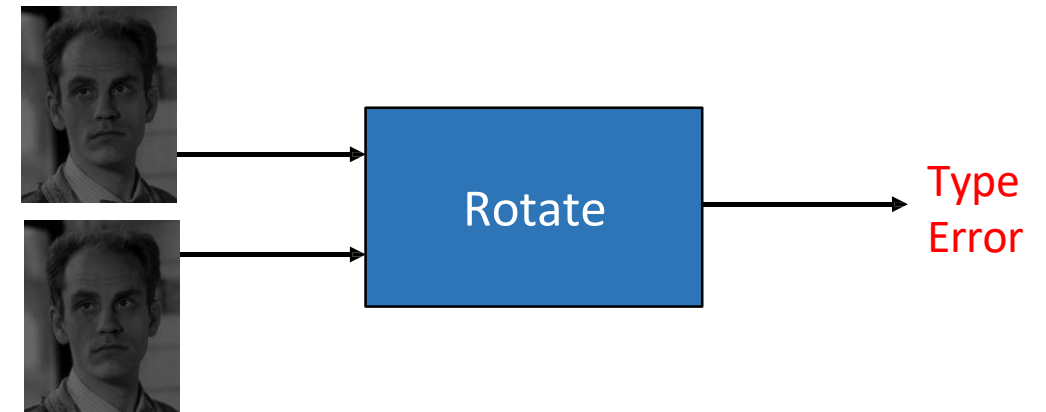
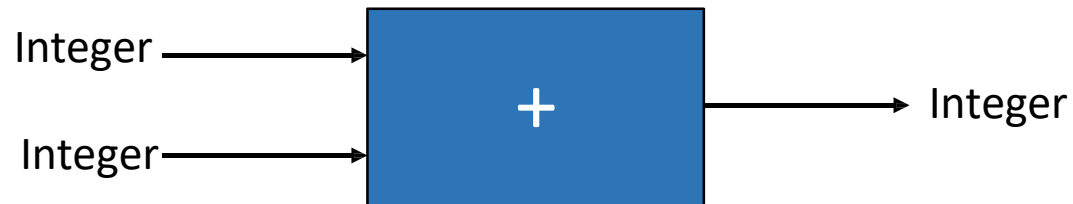
Inputs



Outputs



- **Type:** A group of same *sort of things*.



Structural Way vs. Functional Way

Defining functions to find $x!$ given x

Haskell

```
factorial x = if x>1 then x * factorial (x-1) else 1
```

C

```
int factorial(int x)
{
    int result = 1;
    int index;

    for (index =1; index<= x; index++)
    {
        result = result * index;
    }

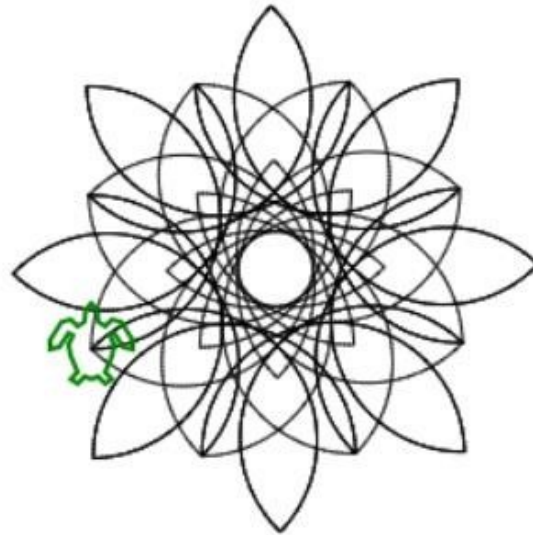
    return result;
}
```

Q: Logo Example

<https://www.transum.org/Software/Logo/>

In Logo programming language, a turtle is moved according to simple commands (functions) and the path the turtle follows forms a drawing.

```
repeat 8 [rt 45 repeat 6 [repeat 90 [fd 2 rt 2] rt 90]]
```



Discussion: What is the paradigm behind Logo?

Entscheidungsproblem and Universality

- *Entscheidungsproblem* (decision problem, 1928)
 - “fundamental problem of mathematical logic” according to David Hilbert
 - Given all the axioms of math, there is an algorithm that can tell if a proposition is provable/valid universally.
 - It is shown to be unsolvable by Alonzo Church and Alan Turing.
 - Church used Lambda Calculus while Turing used Turing Machines to prove it.
- To define the Entscheidungsproblem, a definition for computability is needed.

Entscheidungsproblem and Universality

- **algorithm**: a finite deterministic process
- **computability**: For a computable task, there is at least one algorithm to accomplish.
 - We do not have any restriction on time and memory
- **(total) computable function**: A function which successfully returns a deterministic output for a given input.
 - The domain of a total computable function is the natural numbers.
- **partial computable function**: A function which is not necessarily defined for every input.
 - The domain of a partial computable function is a subset of the natural numbers.

Under no restriction on time and money, should we check a function forever to determine whether it is a partial or total computable function?

Church-Turing Thesis

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO
THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

According to my definition, a number is computable
if its decimal can be written down by a machine.

- A computation on the natural numbers is computable if and only if it can be effectively performed by a Turing Machine.
- General recursive functions, Lambda calculus and Turing machines are equivalent definitions for computability.

Lambda Calculus, a bit

- Lambda calculus can be defined as “the most basic programming language”.
 - applying a function to an argument in prefix form
 - Uses lambda abstraction to define unnamed functions.

$$(\lambda x. x^2 - 1) 2 = 3$$

Substitute 2 for each free occurrence of x.

$$\begin{aligned}(\lambda x. xxz) y &= yyz \\(\lambda x. x) y &= y \text{ (Identity function)} \\(\lambda x. y) z &= y \text{ (Constant function)}\end{aligned}$$

α -Conversion, β -Reduction

- **α -Conversion:** renaming bound variables without changing the meaning of an expression.

$$\lambda x. x + 1 \equiv_{\alpha} \lambda y. y + 1$$

- Useful when different scopes use the same variable name.

$$\begin{aligned} (\lambda x. (\lambda x. x + 5) 2) 3 &\equiv_{\alpha} (\lambda x. (\lambda y. y + 5) 2) 3 \\ &= (\lambda y. y + 5) 2 \\ &= 7 \end{aligned}$$

- **β -Reduction:** evaluating a lambda abstraction by function application.

$$\begin{aligned} &(\lambda x. \lambda y. \lambda z. x^2 + y^2 + z^2) 3 4 5 \\ &= ((\lambda x. \lambda y. \lambda z. x^2 + y^2 + z^2) 3) 4 5 \\ &= (\lambda y. \lambda z. 9 + y^2 + z^2) 4 5 \\ &= (\lambda z. 9 + 16 + z^2) 5 \\ &= 9 + 16 + 25 = 50 \end{aligned}$$

- A variable is bound if it is introduced by a λ .
- A variable is free if it is not bound by any surrounding λ .
- In β -reduction, only free occurrences of the variable are substituted.

Boolean Logic in the Lambda Calculus

- Church Booleans and their functions
 - $true = (\lambda x. \lambda y. x)$: The function that selects first parameter
 - $false = (\lambda x. \lambda y. y)$: The function that selects second parameter
 - $ifelse = \lambda p. \lambda a. \lambda b. p\ a\ b$: If true return first, else return second.

$$(\lambda p. \lambda a. \lambda b. p\ a\ b)\ false\ 1\ 2 = false\ 1\ 2 = (\lambda x. \lambda y. y)\ 1\ 2 = 2$$

$$(\lambda p. \lambda a. \lambda b. p\ a\ b)\ true\ 1\ 2 = true\ 1\ 2 = (\lambda x. \lambda y. x)\ 1\ 2 = 1$$

- $and = \lambda p. \lambda q. p\ q\ p$
- $or = \lambda p. \lambda q. p\ p\ q$
- $not = \lambda p. \lambda a. \lambda b. p\ b\ a$

Boolean Logic in the Lambda Calculus

and true false

$$\begin{aligned} &= (\lambda p. \lambda q. p \ q \ p) \text{true} \ \text{false} \\ &= \text{true} \ \text{false} \ \text{true} \\ &= (\lambda x. \lambda y. x) \ \text{false} \ \text{true} \\ &= \text{false} \end{aligned}$$

and false true

$$\begin{aligned} &= (\lambda p. \lambda q. p \ q \ p) \ \text{false} \ \text{true} \\ &= \text{false} \ \text{true} \ \text{false} \\ &= (\lambda x. \lambda y. x) \ \text{false} \ \text{true} \\ &= \text{false} \end{aligned}$$

or true false

$$\begin{aligned} &= (\lambda p. \lambda q. p \ p \ q) \ \text{true} \ \text{false} \\ &= \text{true} \ \text{true} \ \text{false} \\ &= (\lambda x. \lambda y. x) \ \text{true} \ \text{false} \\ &= \text{true} \end{aligned}$$

not true

$$\begin{aligned} &= (\lambda p. \lambda a. \lambda b. p \ b \ a) \ \text{true} \\ &= \lambda a. \lambda b. \text{true} \ b \ a \\ &= \lambda a. \lambda b. b \\ &= \text{false} \end{aligned}$$

PLs for the course...



The Haskell Programming Language

<http://www.haskell.org/>

- First defined in 1990, named after Haskell B. Curry.
- There are many implementations and interpreters of Haskell.
 - Glasgow Haskell Compiler interactive (GHCi) is the standard.
 - HackageDB database and Cabal packaging infrastructure are used in GHCi as default.



Haskell B. Curry
(1900-1982)

Definitions

- A **functional program** consists of a series of **definitions**.

name :: type

name = expression

```
1  x1 :: Integer
2  x1 = 5
3
4  x2 :: Float
5  x2 = 3.5
6
7  x3 :: String
8  x3 = "Hello World!"
9
10 x4 :: Integer
11 x4 = x1 + round x2
```

```
PS C:\Functional_Programming\Haskell_codes> ghci .\first.hs
GHCi, version 9.2.8: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main                ( first.hs, interpreted )
Ok, one module loaded.
ghci> x3
"Hello World!"
```

```
ghci> x1 + x2

<interactive>:3:6: error:
    * Couldn't match expected type `Integer' with actual type `Float'
    * In the second argument of `(+)', namely `x2'
      In the expression: x1 + x2
      In an equation for `it': it = x1 + x2
```

Function Definitions

- A function definition specifies the types for parameters and the outputs.

name :: input_types -> output_types
name formal_parameters = expression

```
sqr :: Integer -> Integer  
sqr x = x*x
```

```
ghci> sqr 5  
25
```



- For multiple formal parameters:

name :: t1 -> t1 -> t2 -> t3 -> ... -> tk -> output

- Function composition:

```
poweroffour :: Integer -> Integer  
poweroffour = sqr . sqr
```

- Infix evaluation

```
ghci> 1 `sumOfSquares` 2  
5
```


Programming without Side-Effects

- Variables in programming languages using imperative programming are generally like some place to store the values. The current situation of the collection of variables is called the **state**.
- Methods changing the state could cause side-effects.
- **Haskell variables could not be changed. There is no state in Haskell.**
 - **Monad:** Doing I/O operations, inter-operating with other programming languages can be done in Haskell. The side-effects are allowed in a controlling way by using monads.
- Haskell programs are easily parallelized since there is no state to be shared between different threads.

Tests, Properties, Proofs

- cabal install QuickCheck –lib
- We can test a specific case or run a property-based test on random inputs.
 - It does not cover all the cases.
 - It is not the complete proof.
- Using Haskell, proving functions by investigating properties will be easier.

```
import Test.QuickCheck

sqr :: Integer -> Integer
sqr x = x*x

sumOfSquares :: Integer -> Integer -> Integer
sumOfSquares x y = sqr x + sqr y

sumOfSquares_2 :: Integer -> Integer -> Integer
sumOfSquares_2 x y = sqr (x+y) - 2*x*y

singleinput_test :: Bool
singleinput_test = sumOfSquares_2 3 5 == sumOfSquares 3 5

propertybased_test :: Integer -> Integer -> Bool
propertybased_test x y = sumOfSquares_2 x y == sumOfSquares x y
```

Basic Types

- Haskell contains and (&&), or (||) and not (not) **Bool** operators in the **Prelude** (standard module).

```
exor :: Bool -> Bool -> Bool
exor True x = not x
exor False x = x
```

- Float**, **Integer** (fixed-precision), **Int** (arbitrary-precision) types are used to represent numerical data
- Chars** are written in single quotes.

```
ghci> fromEnum 'a'
97
ghci> fromEnum 'A'
65
```

```
toUppercase :: Char -> Char
toUppercase ch = toEnum (fromEnum ch + (fromEnum 'A' - fromEnum 'a'))
```

- Strings** are written in double quotes. They can be joined with «++» operator.

```
ghci> "I\nlove\nYou\n"
"I\nlove\nYou\n"
ghci> putStr "I\nlove\nYou\n"
I
love
You
```

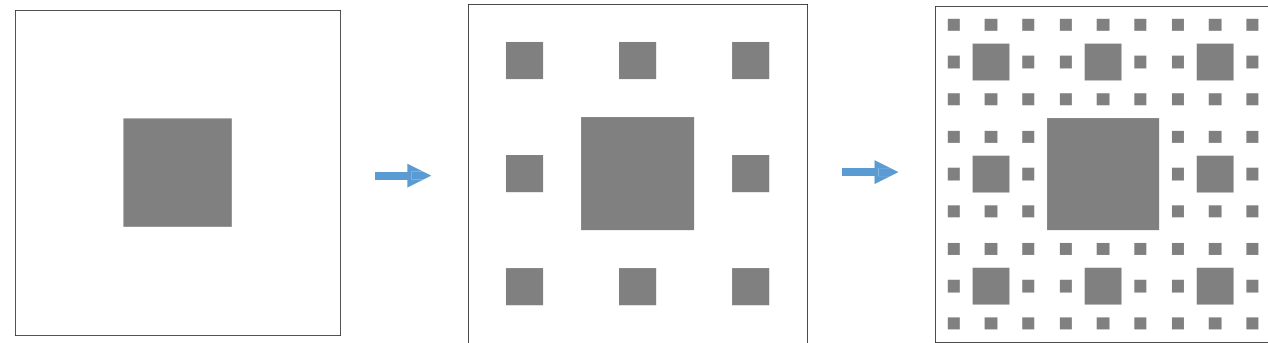
```
ghci> show (2+3) ++ show (2+3) ++ show (2+3)
"555"
ghci> (read "55") :: Int
55
```

Recursion

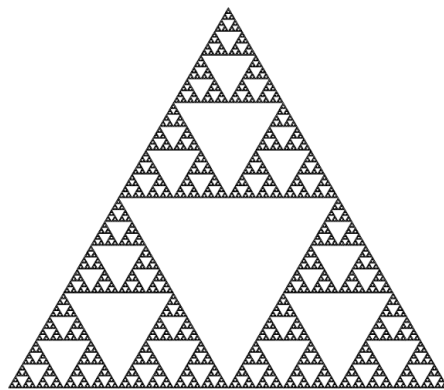


What is Recursion?

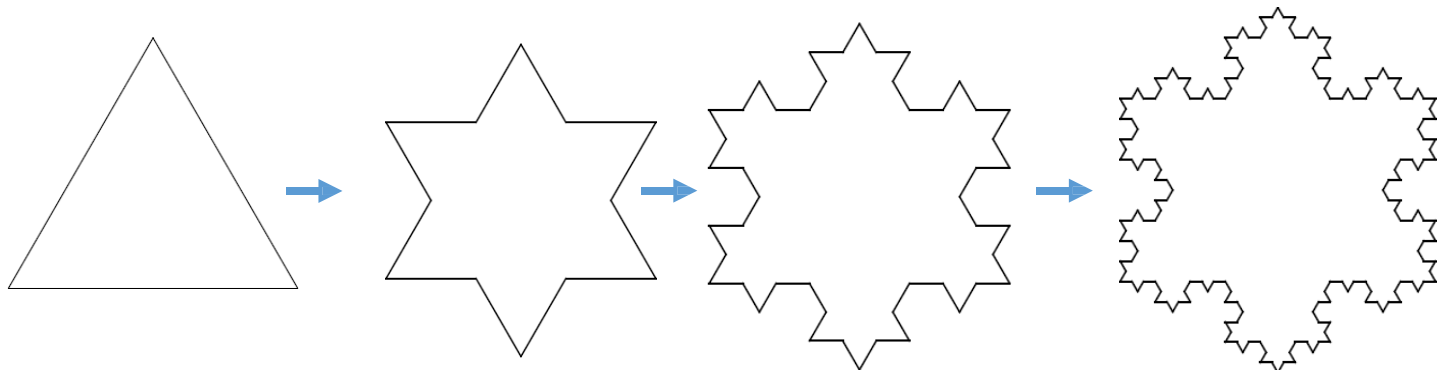
- A recursion is repetition of a task in a **self-similar** way.
- **Fractals**: A geometrical object, whose structure is repeated recursively in different scales.



Sierpiński's carpet



Sierpiński's triangle



Koch's snowflake

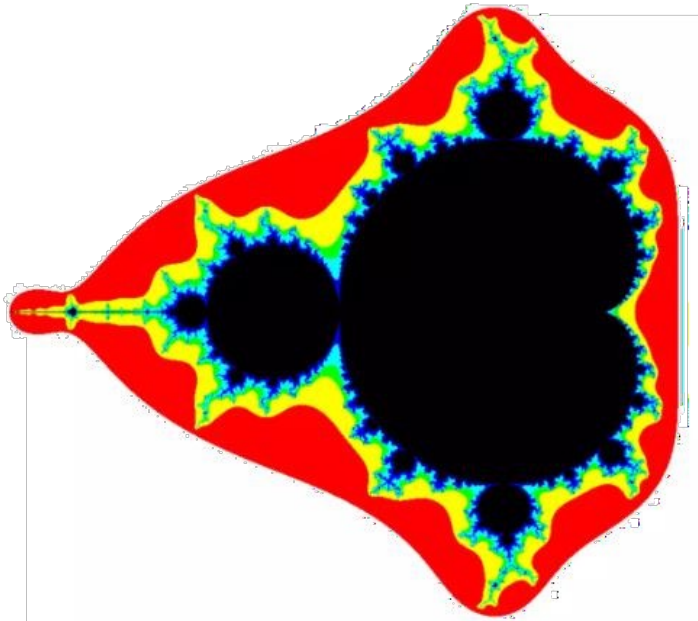
The Mandelbrot Set

- For the complex number c , and initial condition $z_0 = 0$, a sequence of complex numbers could be defined as

$$z_{n+1} = z_n^2 + c$$

- According to different «escape times», a different color is selected for each pixel.
 - Red: z_5 , Yellow: z_8 , Green: z_{12} etc.

<https://bertbaron.github.io/mandelbrot>



Primitive Recursive Function

According to Gödel, the class of primitive recursive functions involves the functions from the following cases.

For the function $f(x_1, \dots, x_n)$,

- **The Zero Rule:** $f(x_1, \dots, x_n) = 0$
- **The Successor Rule:** $f(x_1, \dots, x_n) = x_i + 1$
- **The Projection Rule:** $f(x_1, \dots, x_n) = x_i$
- **The Composition Rule:** $f(x_1, \dots, x_n) = h(g_1(x_1, \dots, x_n), \dots, g_k(x_1, \dots, x_n))$ where $h, g_1 \dots g_n$ are primitive recursive.
- **The Recursion Rule:**

$$\begin{aligned} f(x_1, \dots, x_{n-1}, 0) &= g(x_1, \dots, x_{n-1}) \\ f(x_1, \dots, x_{n-1}, m + 1) &= h(x_1, \dots, x_{n-1}, m, f(x_1, \dots, x_{n-1}, m)) \end{aligned}$$

where h and g are primitive recursive.

PRF Examples

$$f(x) = x + 5$$

Let $S(x) = x + 1$.

$S(x)$ is PRF. (Successor Rule)

$S(S(x))$ is PRF. (Composition Rule)

$S(S(S(S(S(x)))))) = x + 5$ is PRF

$$f(x, y) = x + y$$

Let $G(x, y, z) = z$.

$G(x, y, z)$ is PRF. (Projection Rule)

Let $S(x) = x + 1$.

$S(x)$ is PRF. (Successor Rule)

$h(x, y, z) = S(G(x, y, z)) = z + 1$ is PRF. (Composition Rule)

Repeating the $h(x, y, z)$ function y times:

$$\begin{aligned} f(x, 0) &= x \\ f(x, n+1) &= S(G(x, n, \underbrace{f(x, n)}_y)) \end{aligned}$$

General Recursive Function

Gödel added a sixth axiom to the previously defined five axioms.

μ -operator (unbounded search): Least number with a given property.

Formally, for the function $g(x_1, \dots, x_n, y)$, $f(x_1, \dots, x_n) = \mu y [g(x_1, \dots, x_n, y) = 0]$ is the function which returns the least value of y for the input $[a_1, \dots, a_n]$.

General Recursive Function: For the function $f(x_1, \dots, x_n)$ to be general recursive either

- $f(x_1, \dots, x_n)$ should be primitive recursive, or
- $f(x_1, \dots, x_n) = \mu y [g(x_1, \dots, x_n, y) = 0]$ and the function g is a general recursive function.

<https://www.youtube.com/watch?v=iVuSQosYZgk>

General Recursive Function

Ex: $g(x) = \mu y[f(x_1, \dots, x_n, y) = 0]$ where $f(x, y) = \begin{cases} 0 & y > x \text{ and } y = 2 \\ 1 & \text{otherwise} \end{cases}$

$$f(0,2) = 0 \rightarrow g(0) = 2$$

$$f(1,2) = 0 \rightarrow g(1) = 2$$

$$f(2,2) = 1 \rightarrow \text{No output for } g(2) \text{ (divergence)}$$

Since $f(x, y)$ is a primitive recursive function, general recursive function.

All general recursive functions are **partial computable** functions.

Lambda Calculus & Recursion

- Integers could be represented recursively in lambda calculus.
- «0» is defined as a function that takes in two variables and outputs the second input.

$$0 \equiv \lambda sz. z$$

- The other natural numbers are defined **recursively** using the Successor function.
- A **Successor** function could be defined as

$$S = (\lambda wyx. y(wyx))$$

Using this Successor function:

$$\begin{aligned} 1 &= (\lambda sz. s(z)) \\ 2 &= (\lambda sz. s(s(z))) \\ 3 &= (\lambda sz. s(s(s(z)))) \end{aligned}$$

Successor of 0

$$\begin{aligned} S(0) &= (\lambda wyx. y(wyx))(\lambda sz. z) \\ &= (\lambda yx. y((\lambda sz. z)yx)) \\ &= (\lambda yx. y((\lambda z. z) x)) \\ &= (\lambda yx. y(x)) \\ &= (\lambda sz. s(z)) \\ &= 1 \end{aligned}$$

Thus, any natural number could be defined as $S \dots S(0)$.

The Y-Combinator

Y is a function that takes a function f , and returns $f (f (...))$.

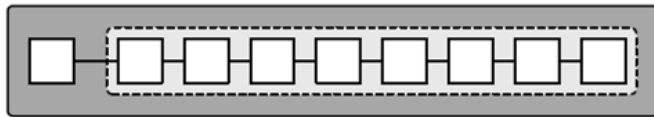
$$Y = \lambda f. (\lambda x. f(x x))(\lambda x. f(x x))$$

For a previously defined function H ,

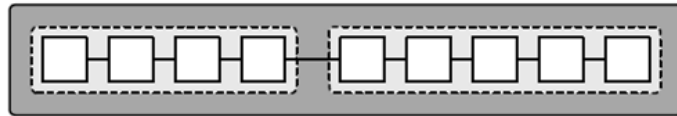
$$\begin{aligned} Y H &= \lambda f. (\lambda x. f(x x))(\lambda x. f(x x)) H \\ &= (\lambda x. H(x x))(\lambda x. H(x x)) \\ &= H \left((\lambda x. H(x x))(\lambda x. H(x x)) \right) \\ &= H \left((\lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))) H \right) \\ &= H (Y H) \end{aligned}$$

Recursion in Data & Code Structures

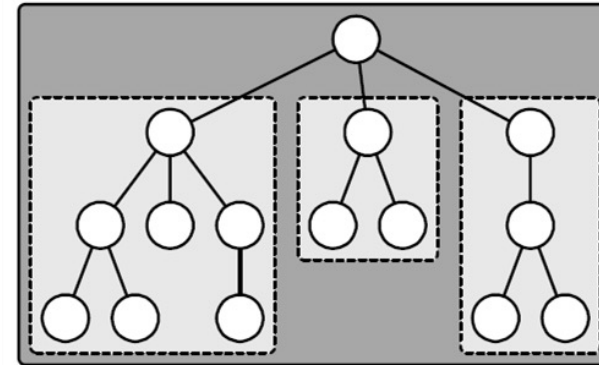
Lists and trees could be decomposed recursively.



A list is a single element plus another list.



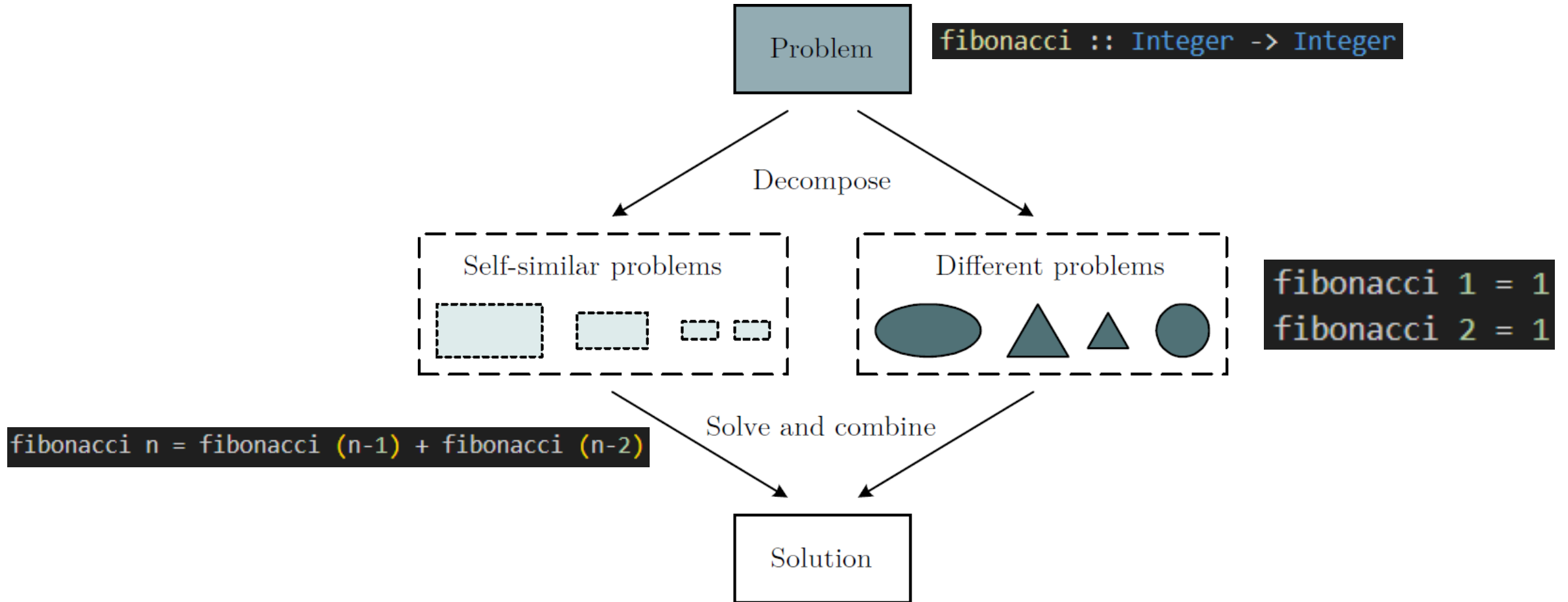
A list is collection of several lists.



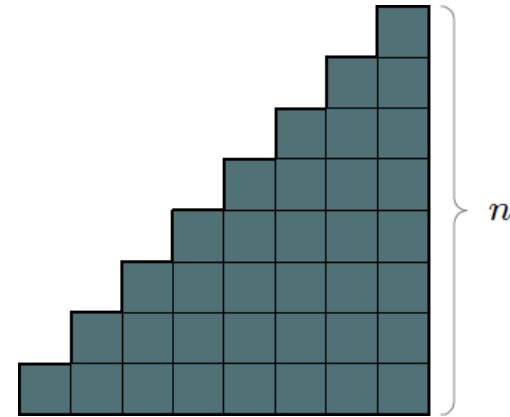
A tree contains subtrees connected to parent node.

Similarly a problem could be divided into self-similar parts to the original problem and different parts to be solved.

Problem Decomposition



Problem Decomposition

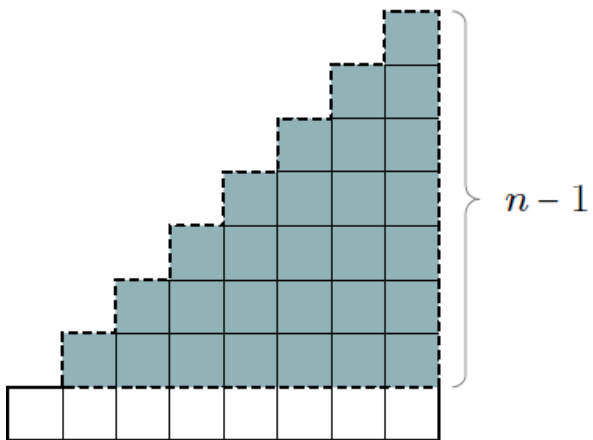


Problem: Calculate the sum of the first n positive integers

Recursive Solution 1:

```
sum_first_naturals :: Integer -> Integer
sum_first_naturals 1 = 1
sum_first_naturals n = sum_first_naturals (n-1) + n
```

$$S(n) = S(n-1) + n$$

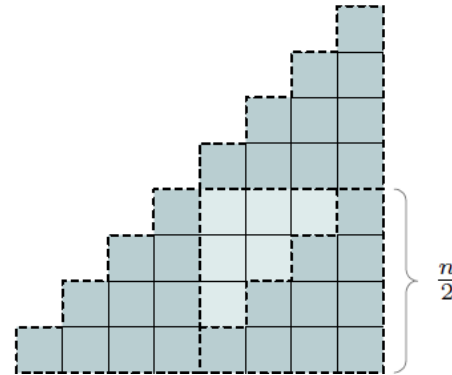


Recursive Solution 2:

```
sum_first_naturals2 :: Integer -> Integer
sum_first_naturals2 1 = 1
sum_first_naturals2 2 = 3
sum_first_naturals2 n
  | even n = 3 * sum_first_naturals2 (div n 2) + sum_first_naturals2 ((div n 2) - 1)
  | otherwise = 3 * sum_first_naturals2 (div (n-1) 2) + sum_first_naturals2 (div (n+1) 2)
```

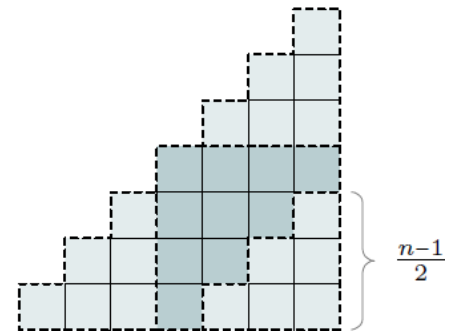
$$S(n) = 3S\left(\frac{n}{2}\right) + S\left(\frac{n}{2} - 1\right)$$

for n even



$$S(n) = 3S\left(\frac{n-1}{2}\right) + S\left(\frac{n+1}{2}\right)$$

for n odd



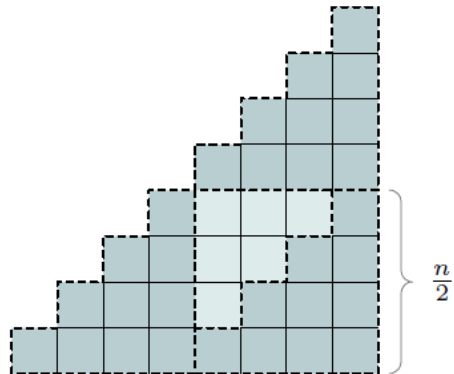
Guards

Guard is a Boolean expression that must be true to continue from that branch.

```
sum_first_naturals2 :: Integer -> Integer
sum_first_naturals2 1 = 1
sum_first_naturals2 2 = 3
sum_first_naturals2 n
  | even n = 3* sum_first_naturals2 (div n 2) + sum_first_naturals2 ((div n 2)-1)
  | otherwise = 3* sum_first_naturals2 (div (n-1) 2) + sum_first_naturals2 (div (n+1) 2)
```

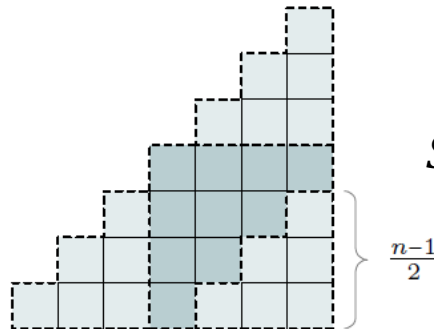
$$S(n) = 3S\left(\frac{n}{2}\right) + S\left(\frac{n}{2} - 1\right)$$

for n even



$$S(n) = 3S\left(\frac{n-1}{2}\right) + S\left(\frac{n+1}{2}\right)$$

for n odd



$$S(n) = \begin{cases} 1, & n = 1 \\ 3, & n = 2 \\ 3S\left(\frac{n}{2}\right) + S\left(\frac{n}{2} - 1\right), & n > 2 \text{ and even} \\ 3S\left(\frac{n-1}{2}\right) + S\left(\frac{n+1}{2}\right), & n > 2 \text{ and odd} \end{cases}$$

Guards

A guard could also be written as an **if-then-else** structure.

```
maxThree :: Integer -> Integer -> Integer -> Integer
maxThree x y z
  | x >= y && x >= z = x
  | y >= z = y
  | otherwise = z

maxThree2 :: Integer -> Integer -> Integer -> Integer
maxThree2 x y z = if x >= y && x >= z then x else if y >= z then y else z
```

Errors could also be defined under a guard.

```
reciprocal :: Float -> Float
reciprocal x
  | x == 0 = error "division by zero"
  | otherwise = 1.0 / x
```

```
between :: Integer -> Integer -> Integer -> Bool
between x y z
  | y >= x && z >= y || y >= z && x >= y = True
  | otherwise = False

middleNumber :: Integer -> Integer -> Integer -> Integer
middleNumber x y z
  | between y x z = x
  | between x y z = y
  | otherwise = z
```

```
data Move = Rock |
  Paper |
  Scissors
  deriving (Eq, Show)

wins :: Move -> Move -> Move
wins x y
  | (x == Paper && y == Rock) || (y == Paper && x == Rock) = Paper
  | (x == Paper && y == Scissors) || (y == Paper && x == Scissors) = Scissors
  | (x == Rock && y == Scissors) || (y == Rock && x == Scissors) = Rock
  | x == y = x
```

Local Definitions

- Local functions and expressions could be defined under `where` block.

```
middleNumber :: Integer -> Integer -> Integer -> Integer
middleNumber x y z
  | between y x z = x
  | between x y z = y
  | otherwise = z
  where
    between :: Integer -> Integer -> Integer -> Bool
    between x y z
      | y >= x && z >= y || y >= z && x >= y = True
      | otherwise = False
```

- The values for local definitions are calculated only if they are needed.
- To make a definition local to an expression we could use `let` keyword.

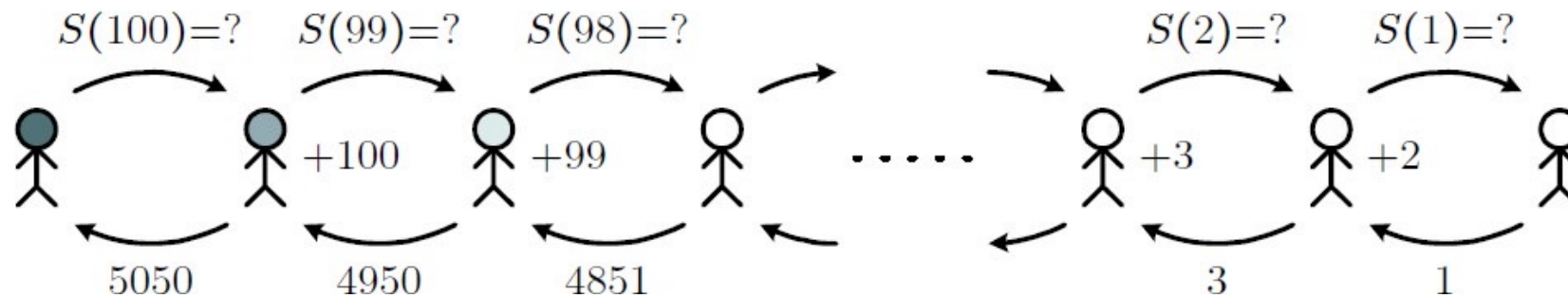
```
let x = 5 in x^2 + 2*x + 1
```

Back to the recursions...

- A recursive algorithm should be proven by [induction](#).

Base case: Verify that the formula is valid for the smallest value of n_0 .

Inductive step: First assume that the formula is true for some general value n , than it should also hold for $n + 1$.



Types of Recursion

- Linear Recursion

The method calls itself only once. However, the recursive call is not the last operation carried in recursion.

$$f(n) = \begin{cases} 1, & n = 1 \text{ or } n = 2 \\ \left\lfloor \phi f(n-1) + \frac{1}{2} \right\rfloor, & \text{otherwise} \end{cases}$$

- Tail Recursion

The method also calls itself once. However they do not manipulate the result of the recursive call.

$$f(n, a, b) = \begin{cases} b, & n = 1 \\ f(n-1, a+b, a), & n > 1 \end{cases}$$

Types of Recursion

- Multiple Recursion

The method calls itself several times. Typically used in divide&conquer algorithms.

$$S(n) = \begin{cases} 1, & n = 1 \\ 3, & n = 2 \\ 3S\left(\frac{n}{2}\right) + S\left(\frac{n}{2} - 1\right), & n > 2 \text{ and even} \\ 3S\left(\frac{n-1}{2}\right) + S\left(\frac{n+1}{2}\right), & n > 2 \text{ and odd} \end{cases}$$

- Mutual Recursion

A set of functions call each other in a circular manner.

$$A(n) = \begin{cases} 0, & n = 1 \\ A(n-1) + B(n-1), & n > 1 \end{cases} \quad B(n) = \begin{cases} 1, & n = 1 \\ A(n-1), & n > 1 \end{cases}$$

- Nested Recursion

The argument of the recursive function is defined through another recursive call.

$$f(n, s) = \begin{cases} 1 + s, & n = 1 \text{ or } n = 2 \\ f(n-1, s + f(n-2, 0)), & n > 2 \end{cases}$$

Ex: Recursive Quine in C

- A quine program, or simply a quine, is a program designed to output its own source code when executed.
- The traditional method for creating a self-replicating program involves two steps:
 1. Define a string variable that includes a placeholder for self-referencing.
 2. Print this string, inserting the string itself into the placeholder.

```
#include <stdio.h>
int main(){
char*s="#include <stdio.h>%cint
main(){%cchar*s=%c%s%c;%cprintf(s,10,10,34,s,34,10);return 0;}";
printf(s,10,10,34,s,34,10);return 0;}
```

```
#include <stdio.h>
int main(){
char*s="#include <stdio.h>%cint main(){%cchar*s=%c%s%c;%cprintf(s,10,10,34,s,34,10);return 0;}";
printf(s,10,10,34,s,34,10);return 0;}
```

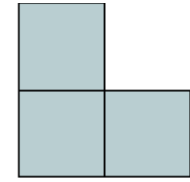
A Recursive Quine: Creates a new code file, which creates another code file...

<https://github.com/iamthememory/superquine>

Ex: Tromino Tiling Problem

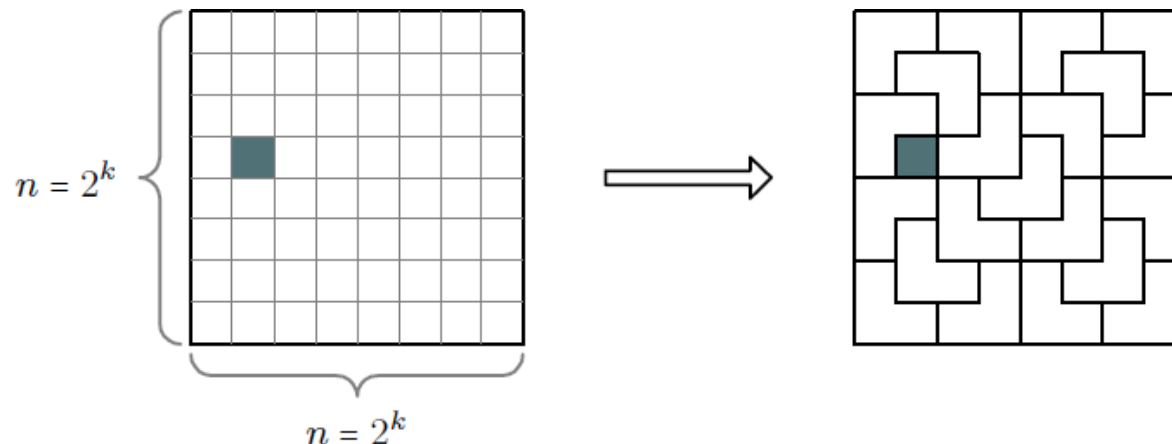


"I" tromino



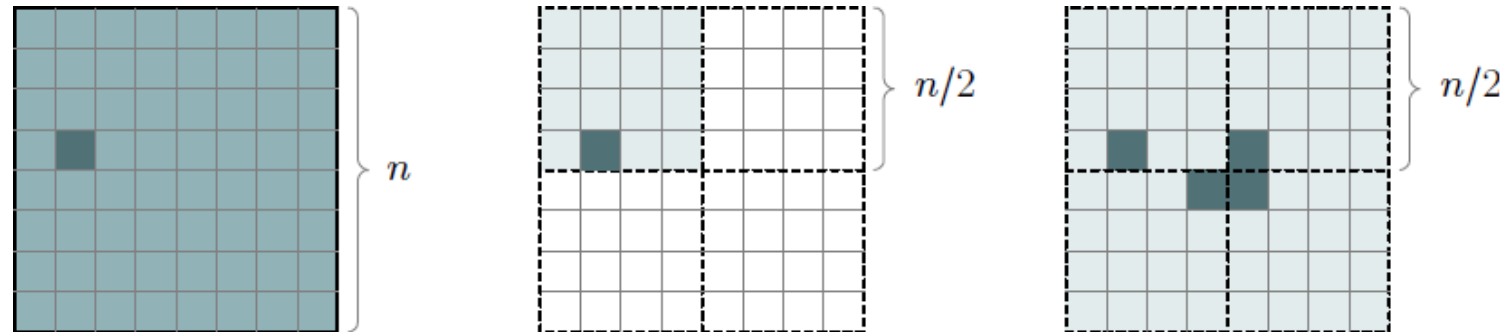
"L" tromino

- A tromino is a shape created by joining three squares of equal size along their edges. Ignoring rotations and reflections, there are only two distinct types of trominoes: the "I" shaped tromino and the "L" shaped tromino.
- The problem involves covering an $n \times n$ square board, where $n \geq 2$ and is a power of two, with "L" shaped trominoes, except for a single "hole" square that cannot be covered. The goal is to place the trominoes so that they cover all other squares on the board without overlapping or covering the hole.



Ex: Tromino Tiling Problem

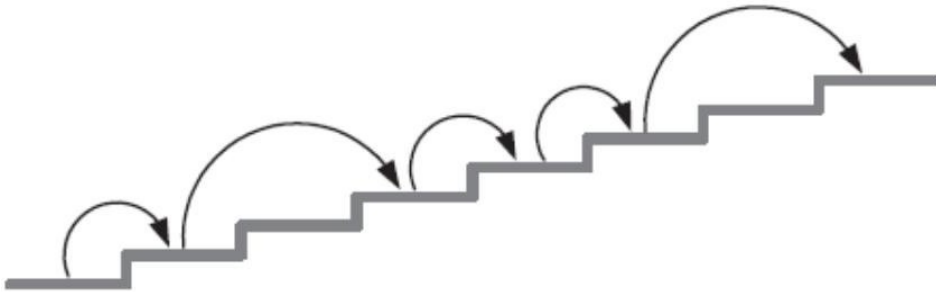
- Decomposition of the tromino tiling problem



- The problem size is defined by n .
- The approach uses a divide-and-conquer strategy for larger boards.
- The board is divided into four smaller $\frac{n}{2} \times \frac{n}{2}$ boards.
- However, only one of these smaller boards contains the initial hole.
- To address this, a single is used to effectively create "holes" in each of the other three smaller boards.

Ex: Staircase Climbing

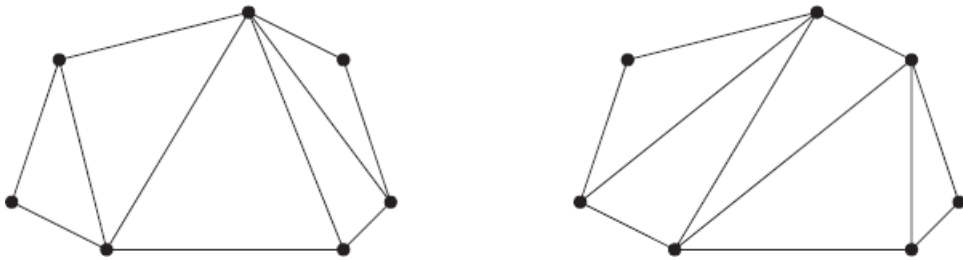
Suppose that, to climb a staircase of n steps, we have two different actions: taking an individual step or leaping two steps. What is the function $f(n)$ which counts the number of ways to reach the top?



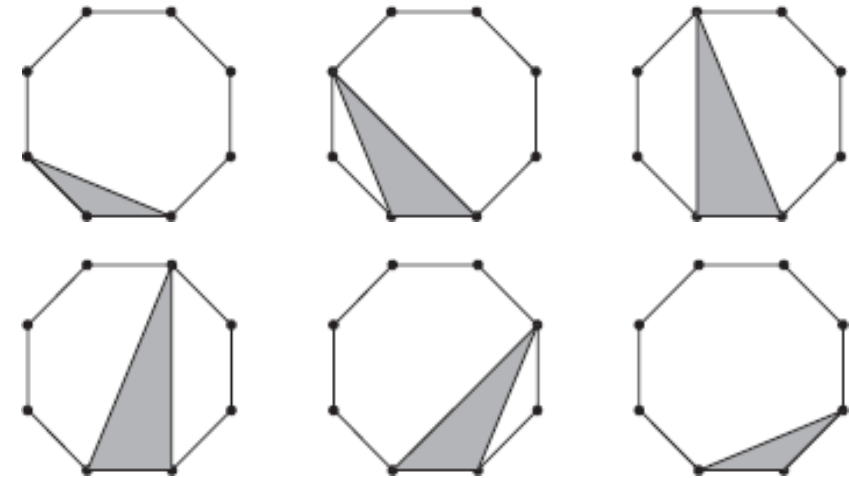
$$f(n) = \begin{cases} 1, & n = 1 \\ 2, & n = 2 \\ f(n-1) + f(n-2), & n = 3 \end{cases}$$

Ex: Convex Polygon Triangulations

A triangulation of a polygon is a division of the polygon into triangles that fully cover its area without overlapping, using only the polygon's vertices as the triangle corners. How can we define a function to calculate the number of possible ways to triangulate a convex polygon with n vertices?



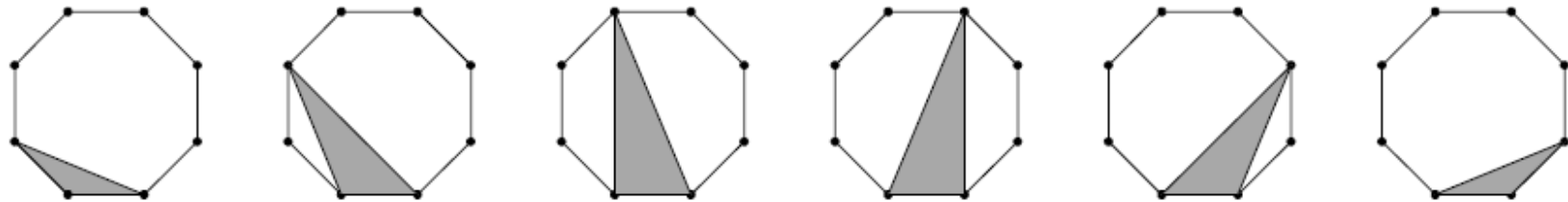
Two possible triangulations of the same polygon



Six possible triangles for the same edge of an octagon

Ex: Convex Polygon Triangulations

Total number of triangulations of an octagon:



$$f(8) = f(2)f(7) + f(3)f(6) + f(4)f(5) + f(5)f(4) + f(6)f(3) + f(7)f(2)$$

$$f(n) = \begin{cases} 1 & \text{if } n=2, \\ \sum_{i=2}^{n-1} f(i) \cdot f(n+1-i) & \text{if } n>2. \end{cases}$$

Recursive Examples from Haskell

Greatest Common Divisor

```
my_gcd :: Integer -> Integer -> Integer
my_gcd x y = if y == 0 then x else my_gcd y (mod x y)

my_gcd2 :: Integer -> Integer -> Integer
my_gcd2 x y
  | y == 0 = x
  | otherwise = my_gcd2 y (mod x y)
```

Factorial

```
factorial :: Integer -> Integer
factorial x = if x > 1 then x * factorial (x-1) else 1

factorial2 :: Integer -> Integer
factorial2 x
  | x > 1 = x * factorial2 (x-1)
  | otherwise = 1
```

Power of a Natural Number

```
pow :: Integer -> Integer -> Integer
pow x y
  | y == 0 = 1
  | otherwise = x * pow x (y - 1)
```

Power, computationally simpler way

```
pow2 :: Integer -> Integer -> Integer
pow2 x y
  | y == 0 = 1
  | even y = sqr (pow x (div y 2))
  | otherwise = x * pow x (y - 1)
  where
    sqr :: Integer -> Integer
    sqr n = n * n
```

Tuples & Lists

Tuple: Fixed number of values of fixed types

```
type ShopItem = (String,Int)

x1 :: ShopItem
x1 = ("Rice", 40)

x2 :: ShopItem
x2 = ("Salt", 10)

x3 :: ShopItem
x3 = ("Beer", 10)
```

List: Arbitrary number of values of the same type

```
[("Rice", 40),("Salt", 10),("Beer", 10)]
```

```
ghci> [("Rice", 40),("Salt", 10),("Beer", 10),4]

<interactive>:3:1: error:
  * No instance for (Num (String, Integer))
    arising from a use of `it'
  * In the first argument of `print', namely `it'
     In a stmt of an interactive GHCi command: print it
```

Ex: The STL Structure

A formal definition for a 3D mesh

- A collection of triangles.

```
solid Mesh
facet
outer loop
vertex 0.0666225 -0.00713973 -0.0520612
vertex 0.0695272 -0.00912108 -0.0509354
vertex 0.0659653 -0.00814601 -0.052367
endloop
endfacet
facet
outer loop
vertex 0.0762163 -0.00201969 -0.0587023
vertex 0.0769302 -0.00441556 -0.0564184
vertex 0.0760299 -0.00791856 -0.0610091
endloop
endfacet
```